

Anatomy of Coding Agents

References

01 O'Reilly, *AI-Native Software Engineering* (early release), "From LLMs to Agents."

02 S. Raschka, "Components of A Coding Agent," *Ahead of AI*, April 2026.

03 S. Raschka, Mini Coding Agent. github.com/rasbt/mini-coding-agent

04 A. Vaswani et al., "Attention Is All You Need," NeurIPS 2017.

05 N. F. Liu et al., "Lost in the Middle: How Language Models Use Long Contexts," 2023.

06 C.-P. Hsieh et al., RULER, 2024; M. Karpinska et al., NoCha, 2024.

07 DeepSeek-AI, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via RL," 2025.

08 S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," ICLR 2023.

09 J. Chaumond, "Tiny Agents: An MCP-Powered Agent in 50 Lines of Code," Hugging Face, 2025.

10 Anthropic, "Building Effective Agents," 2024. anthropic.com/engineering

What we covered last week

Week 2: Spec-Driven Development

The SDLC became a loop

Plan, design, build, test, deploy and maintain now cycle in hours around coding AI.

Specs before code

Agents guess fast and silently. A good spec removes the guesses.

Humans at checkpoints

Plan Mode, stop conditions and the spec interview keep you in control.

Match ceremony to risk

Plan Mode for small tasks; a living spec for features that last.

TODAY **What is inside a coding agent, and why does it behave the way it does?**

Today

01

How LLMs work

02

The model landscape

03

Why LLMs are good at coding

04

Coding agents under the hood

05

Six components of a coding harness

LAB

Lab 3: a mini coding agent

01 How LLMs work

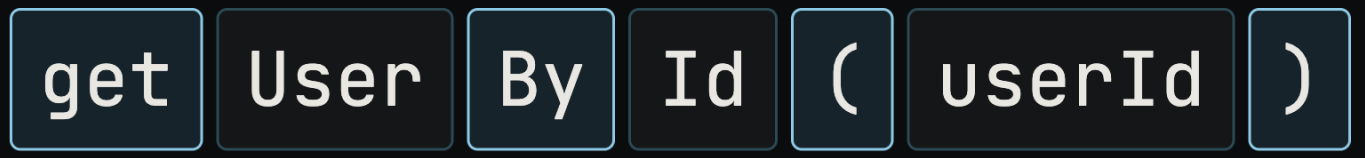
What is an LLM?

A large language model (LLM) is a neural network with billions of parameters, trained on huge amounts of text and code to predict the next token.

It doesn't store its training text. It stores statistical patterns about what comes next.

Models read tokens, not words

A COMMON TOKENIZER SPLITS `getUserById(userId)` ROUGHLY LIKE THIS



≈ 7 tokens

3–4

characters per token in English prose. About ¾ of a word.

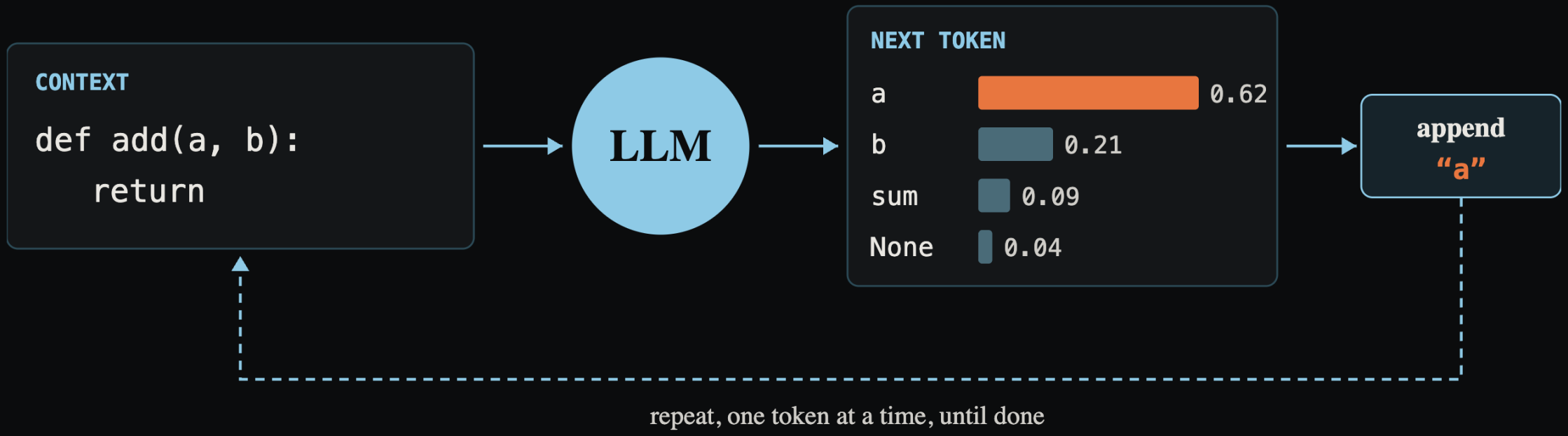
~10

`def add(a, b): return a + b`

4–5k

tokens for a 600-line TypeScript

One token at a time



No planning pass, no going back: early mistakes carry through.

Attention mechanism

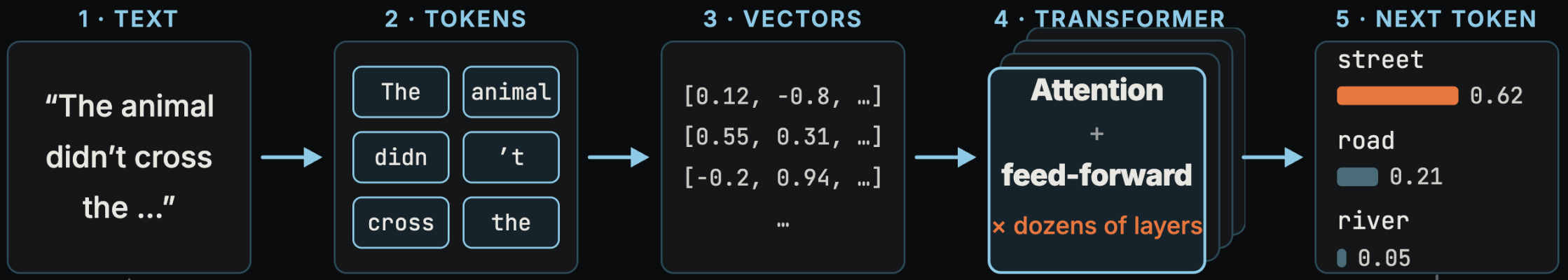


“it” looks at every word and gives the most weight to **“animal”**, the thing it refers to.



Change the last word to “wide” and the most weight moves to **“street”**.

The Transformer



The input text is what the model must continue.

The text is split into small pieces called tokens.

Each token is turned into a list of numbers.

Dozens of layers let the tokens exchange information.

The model scores every possible next token and picks one.

2017

Google’s “Attention Is All You Need” introduced it.

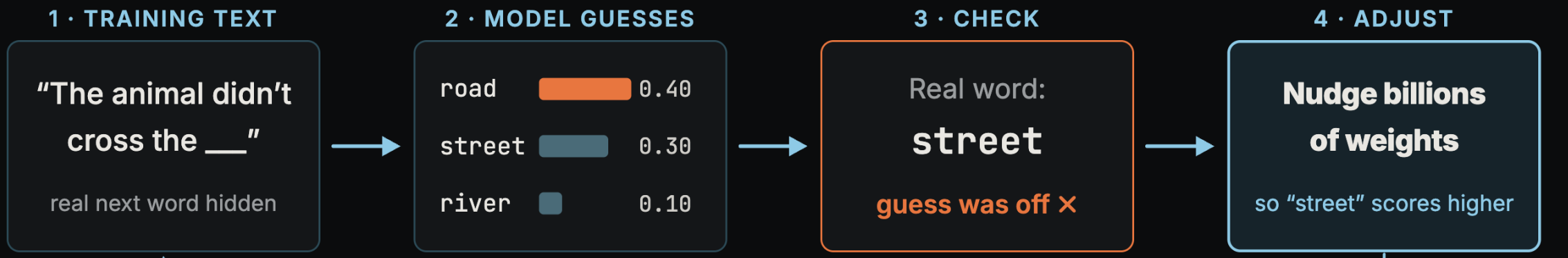
EVERYWHERE

GPT, Claude, Gemini, Llama and Qwen are all Transformers.

WHY IT WON

It reads all tokens in parallel, so it trains at huge scale.

How the model learns



THE WEIGHTS

Everything it learned ends up in these numbers.

FROZEN

Training stops at a **cutoff date**. Nothing new gets in.

THEN POST-TRAINING

Makes it follow instructions. More in section 3.

Post-training with human preference feedback

People compare responses and teach the model which ones they prefer.

01

Generate alternatives



The model produces two responses.

02

Compare responses



A person selects the better response.

03

Learn the preference



A reward model learns patterns in the comparisons.

04

Improve future responses



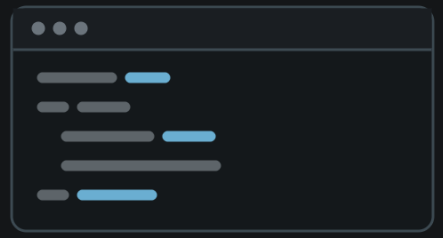
The language model learns to produce responses that receive higher scores.

Human feedback teaches the model what people prefer, but preference does not always guarantee correctness.

Post-training with verifiable feedback

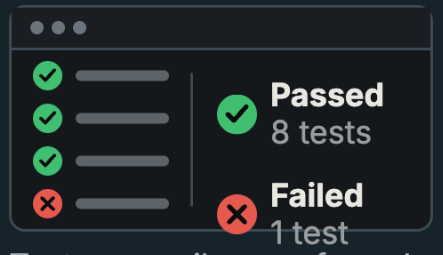
Tests and other automatic checks provide a result that can be measured directly.

01
Attempt the task



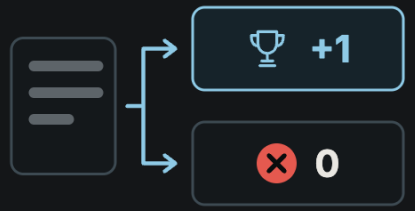
The model writes code or solves a problem.

02
Run the verifier



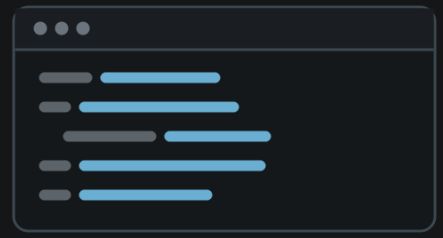
Tests, compilers, or formal checks evaluate the result.

03
Assign the reward



Passing results receive a positive reward. Failing results do not.

04
Learn from the outcome



The model learns strategies that produce verifiable success.

Two kinds of model knowledge

IN-WEIGHTS · PARAMETRIC

Learned during training

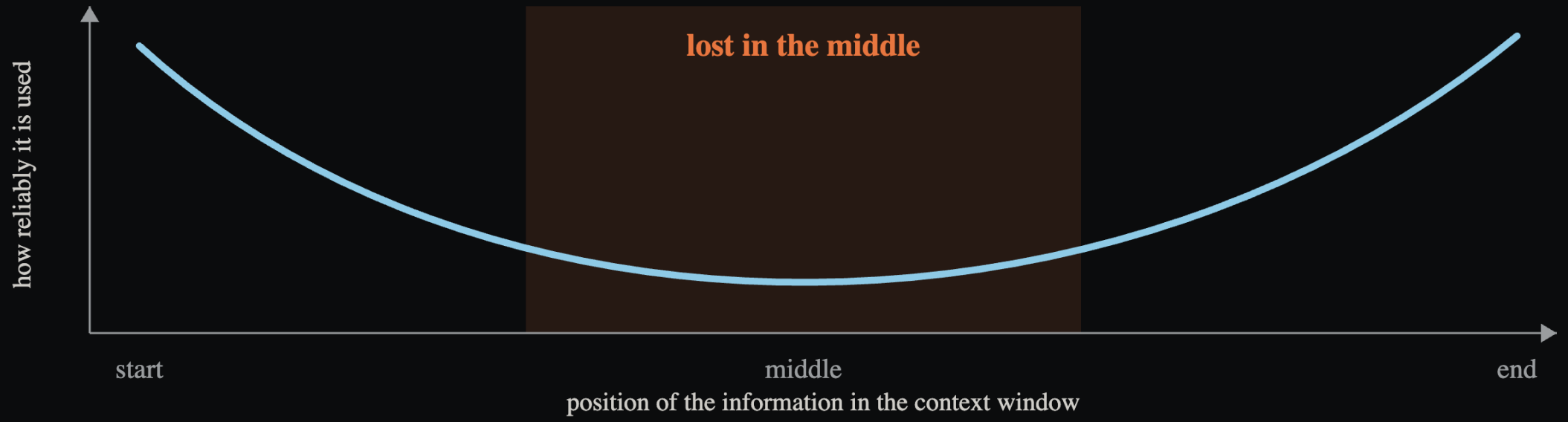
- Always available
- Fixed at the training cutoff
- May be stale or wrong

IN-CONTEXT · NON-PARAMETRIC

Supplied with the request

- Prompt, files, docs, and tool results
- Current and authoritative when trusted
- Uses the context window; not retained

The context window is a budget



SIMPLE LOOKUP

"Needle in a haystack": modern frontier models are solid up to their advertised window.

HARDER TASKS

Multi-step reasoning, tracking entities, conflicting facts: quality drops well before the limit.

One more time: Models are probabilistic

The same prompt can produce different, equally correct outputs.

SAME PROMPT Write a function that checks if a number is even.



RUN 1 · MONDAY

```
def is_even(n):
    return n % 2 == 0
```

RUN 2 · WEDNESDAY

```
def is_even(n: int) -> bool:
    return not n & 1
```

SO TEST BEHAVIOR, NOT STRINGS

- Does it parse as valid JSON?
- Do the tests pass?
- Is the edge case handled?
- Does the linter pass?
- **Not:** does it match yesterday's text?

Even at temperature 0, outputs can vary slightly across runs and hardware.

02 The model landscape

How open is “open source”?

The term is used loosely. It helps to distinguish three levels.

MORE OPEN, MORE CONTROL

MORE CLOSED, HIGHER CEILING

FULLY OPEN

Weights, data, code

Most auditable and reproducible. Usually a lower ceiling.

OPEN WEIGHTS

Weights only

Llama, Mistral, Qwen, DeepSeek. Self-host and fine-tune; training data stays private.

CLOSED

API access

GPT, Claude, Gemini. Can run in your cloud, never on your own servers.

Choose by hard constraints: where data may go, fine-tuning needs, ops effort, vendor lock-in.

Kinds of language models

Different training choices produce different kinds of models.

Pre-trained

Raw foundation models that learn general language patterns from broad internet text.

EXAMPLES

Early GPT, BERT

Fine-tuned

Adapted from base models using smaller, task-specific data.

EXAMPLES

Customer support, chat

Reasoning

Trained with reinforcement learning to spend extra compute on step-by-step logic, math and code before answering.

EXAMPLES

Logic, math, code

Specialized

Built for distinct fields such as medicine or law.

EXAMPLES

BioBERT, legal models

Reasoning models

Chain of thought: each written step conditions the next. Reasoning models do it by default. More tokens, more compute: **test-time compute**.

STANDARD MODEL	
SPEED	Fast
COST	Lower
USE FOR	Routine, well-specified work
EXAMPLE	Reformat JSON, a one-line function

REASONING MODEL	
SPEED	Slower, sometimes much slower
COST	Higher: hidden thinking tokens are billed
USE FOR	Ambiguous, multi-step, high-stakes work
EXAMPLE	A race condition, a schema with many invariants

Frontier models, October 2026

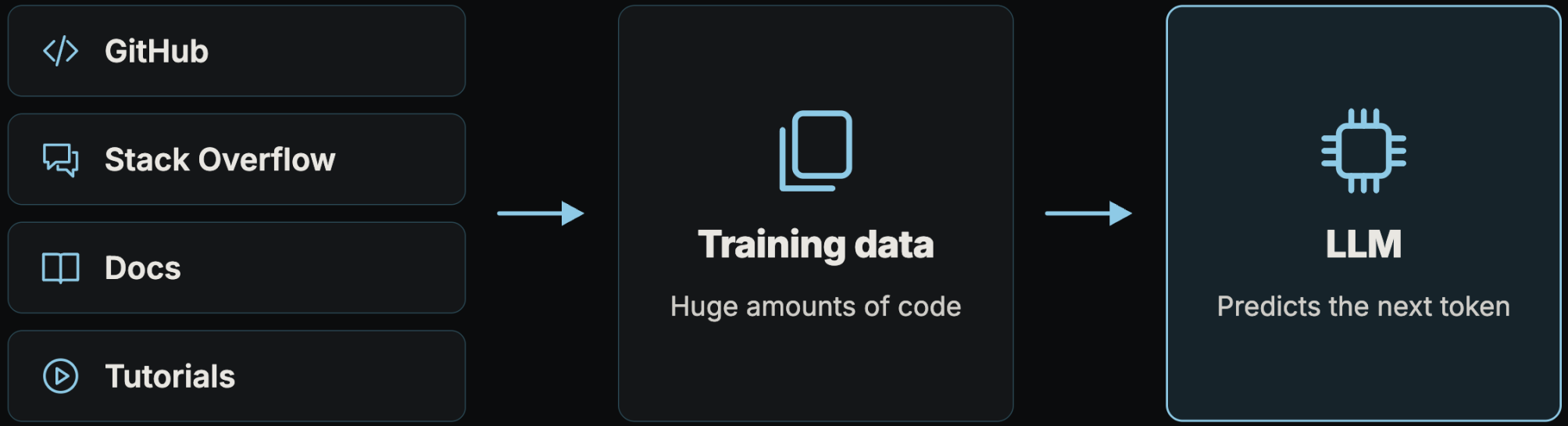
MODEL	CONTEXT
CLOSED MODELS · API ONLY	
GPT-6 Astra	1.05M
Claude Fable 5.1	1M
Gemini 3.8 Flash	~1M
OPEN-WEIGHTS MODELS · CHINESE LABS	
Kimi K3	1M
DeepSeek V4.1 Flash	1M
Qwen3.8	262K
GLM-5.3	–
MiniMax M3	~1M

03 Why LLMs are good at coding

WHY LLMS ARE GOOD AT CODE · 1 OF 4

Seen it millions of times

GitHub, Stack Overflow, docs, tutorials: huge amounts of code in training data.



It runs or it doesn't

Code is strict. A missing bracket breaks it, which gives a clear training signal.

RUNS

```
def add(a, b):
    return a + b
```

It runs

ERROR

```
def add(a, b:
    return a + b
```

SyntaxError: '(' was never closed

Strict rules give a clear training signal: the code works or it does not.

Boilerplate is predictable

CRUD endpoints, test setup and config files repeat in slightly different forms.

THREE ENDPOINTS

```
@app.get("/users")
def list_users():
    return db.users.all()

@app.get("/orders")
def list_orders():
    return db.orders.all()

@app.get("/items")
def list_items():
    return db.items.all()
```

- **CRUD endpoints**
- **Test setup**
- **Config files**

Code and docs together

Source, README, docs and Q&A about the same library, all in training.



Strongest for widely used languages and frameworks. Weaker for an internal framework that exists only inside one company.

04 Coding agents under the hood

Talking to an LLM: the messages array

```
{
  "messages": [
    { "role": "system",
      "content": "You are a code reviewer.
        Focus on security and performance." },
    { "role": "user",
      "content": "Review this function: ..." },
    { "role": "assistant",
      "content": "Line 12 has an injection..." }
  ]
}
```

STATELESS

No memory between calls. Your app resends the full history every time.

SYSTEM PROMPT

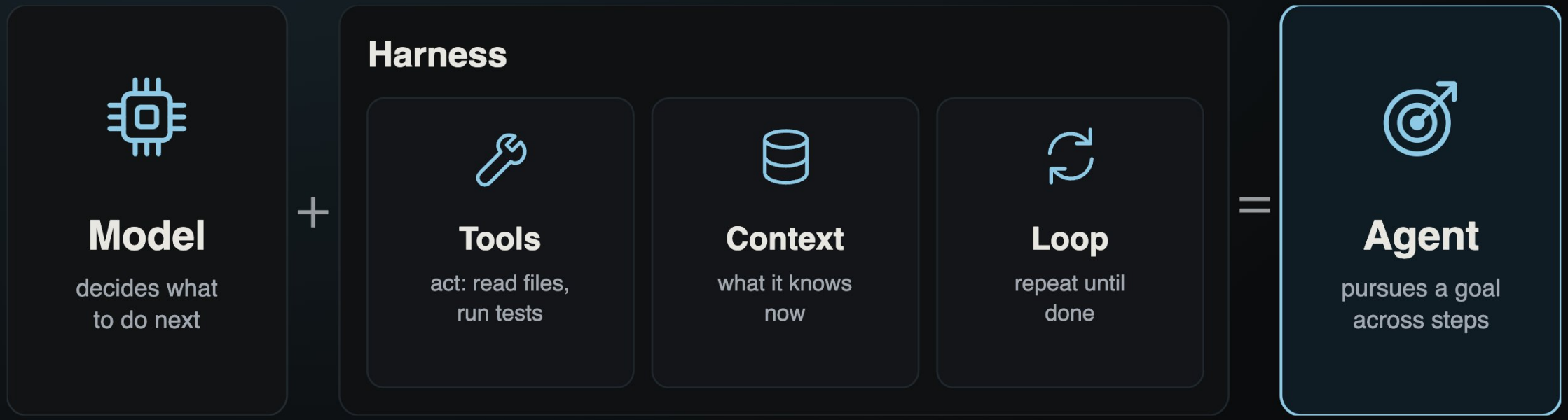
Read first, on every call. Your highest-leverage input.

COST GROWS

Each turn resends everything, so long sessions get slow and expensive.

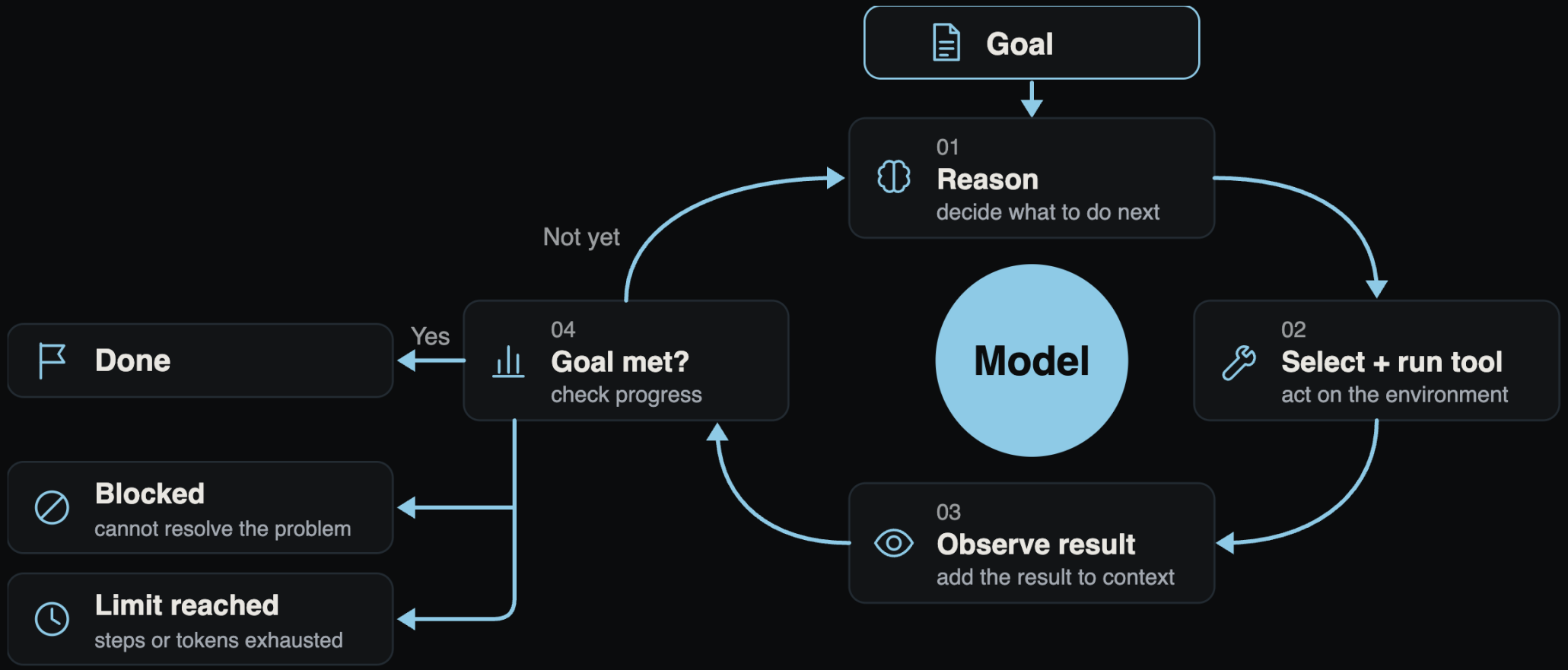
What is an agent?

A language model combined with **tools, context and a loop**—able to pursue a goal over several steps without approval at each step.



The agentic loop

Reason, act and observe—until the goal is reached or the run must stop.



Also called Reason–Act–Observe or Plan–Execute–Reflect.

The agent loop in about ten lines

The model call, tool execution and message history form one repeating control flow.

```
1  messages = [user_message(goal)]

2  while True:
3      response = model_generate(messages, tools)
4      messages.append(response)

5      if not response.tool_calls:
6          return response.content

7      for call in response.tool_calls:
8          result = execute_tool(call)
9          messages.append(tool_result(call.id, result))
```

1 **Call model**
current messages + available tools

2 **Complete?**
no tool calls → return response

3 **Execute tools**
run every requested action

4 **Append + repeat**
add results to history and call again

Frameworks
Google ADK and others implement this loop for you.

Providers format messages differently;
the control flow stays the same.

Memory · retries · streaming build on top of this loop.

ReAct: think, act, observe

Yao et al. (2022) · ICLR 2023

A visible reasoning trace connects each decision to an action and its result.



INTERPRETABILITY
You can read why it did each step.

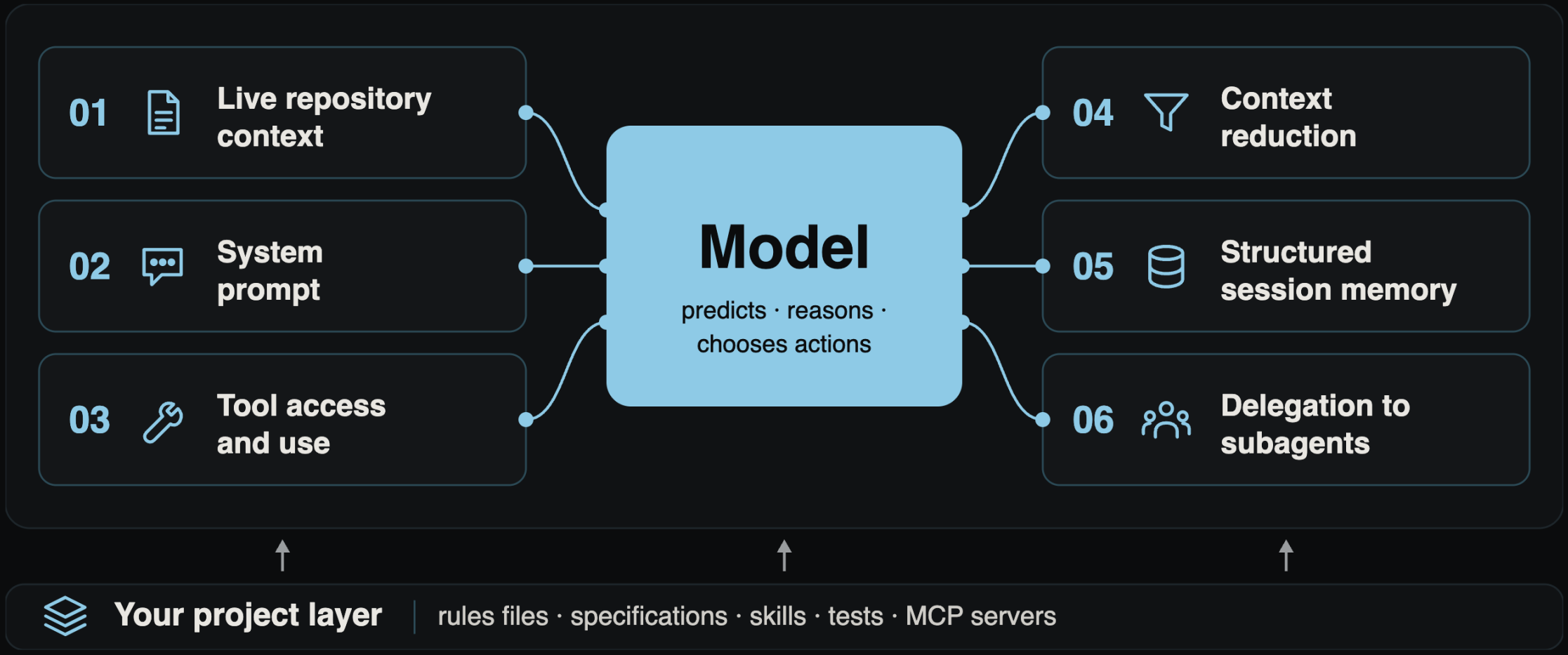
ERROR LOCALIZATION
A bad assumption shows up in the trace.

GROUNDING
Each step sees the real code, not a guess.

05 Six components of a coding harness

SECTION 05

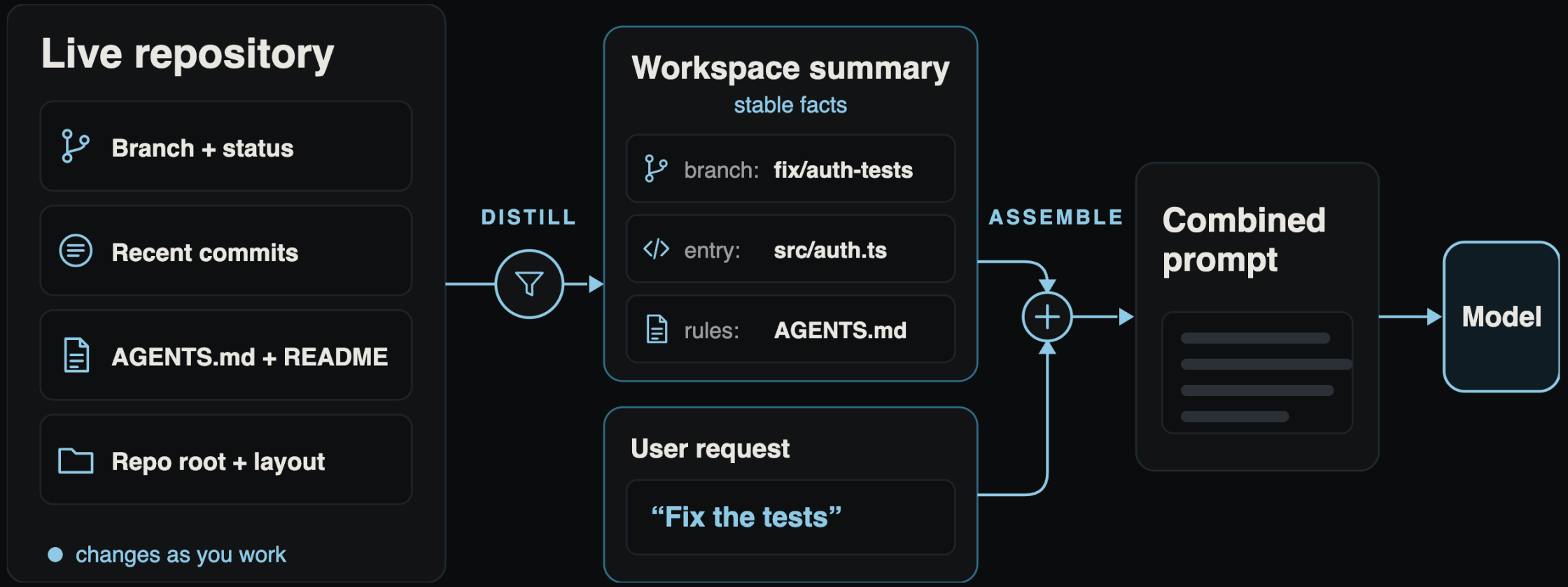
Agent = Model + Harness



COMPONENT 01

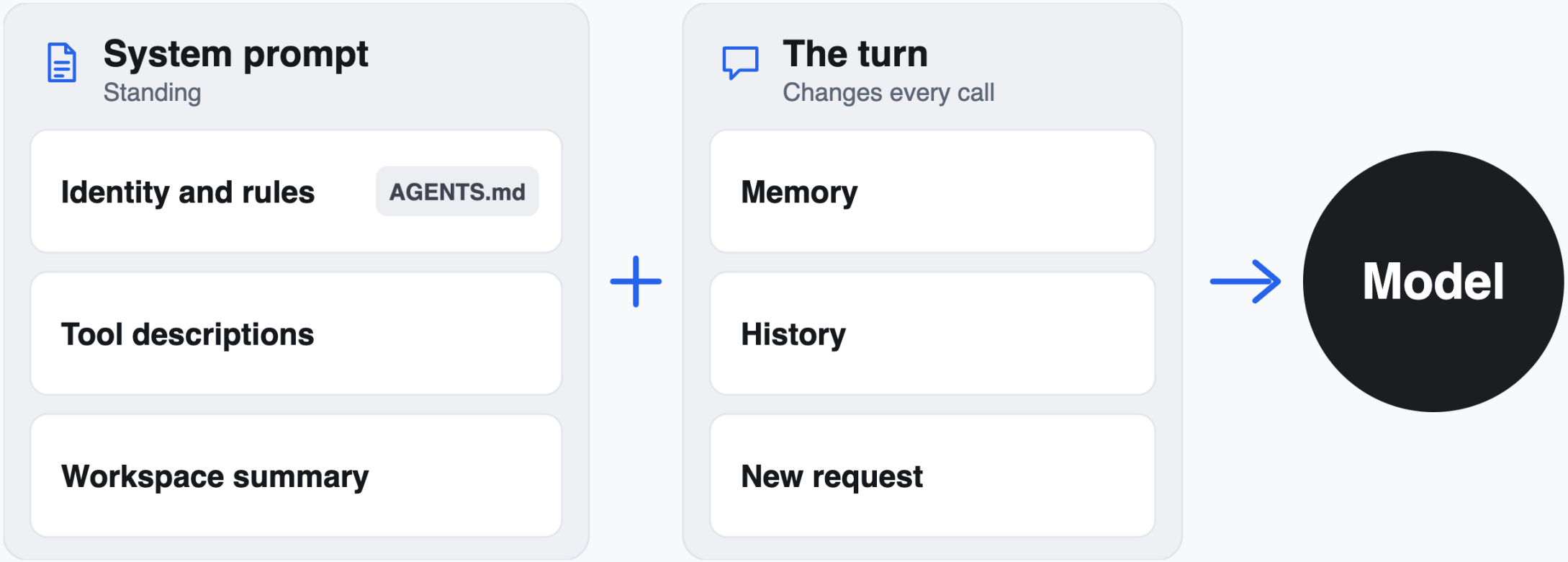
Live repo context

The harness turns a changing repository into stable prompt context.



COMPONENT 02

The system prompt



COMPONENT 03 · CLAUDE CODE'S DEFAULT TOOLS

Tool access and use



Read and search

See files, find code

`Read` · `Glob` · `Grep`



Change files

Change or create files

`Edit` · `Write` · `NotebookEdit`



Run

Tests, git, shell commands

`Bash`



Look things up

Search the web, read pages

`WebSearch` · `WebFetch`



Plan and delegate

Plan first, hand off work

`EnterPlanMode` · `ExitPlanMode` · `Agent`

Source: Anthropic, Claude Code tools reference, Oct 2026

COMPONENT 04

Context reduction

Clip

```
1      > npm test          npm test output · 3,000 lines
2      > vitest run
3      ✓ 124 passed (2.1s)
       …
```



```
1      > npm test          first + last lines …
2      > vitest run
       …
3,000  ✓ 124 passed (2.1s)
```

Dedupe

read utils.py (1)

read utils.py (2)

read utils.py (3)



utils.py, latest only

Compress

T1 | T2 | T3 | T4 | T5 | T6 | T7 | T8



T1–5 summary | T6 | T7 | T8

COMPONENT 05

Session memory

The model retains no state between calls.

Model calls

Fresh invocation each call.

Harness-managed state

Stored outside the model · usually on disk.

Full transcript APPEND

Every request, response and tool output.

`e.g. transcript.json`

1	Request
2	Response
3	Tool output
4	+ New event

Preserves history so a session can resume.

Working memory REWRITE

A small, distilled view of the current state.

`e.g. memory.json`

TASK	Fix failing auth tests
FILES	`src/auth.ts`
NOTES	Refresh token expired

Updated as the task evolves.

COMPONENT 06

Delegation to subagents

Side questions run in parallel while the main agent keeps working.

